

The Future of AI Deployment Is Hybrid

A practical map of where AI models can run, and why the winning designs route across all of it, not one rung.

Most teams still treat AI deployment as a binary: call the vendor's API, or self-host. That framing is already obsolete. The real question is not which one you choose, it is how you route work across a whole spectrum of options, and the teams that see this first will have a genuine edge on cost, control, and speed.

There are roughly seven ways to run an AI model, and they form a ladder. At the top, frontier vendor models, maximum convenience and power, least control over your data. At the bottom, open models you host yourself, maximum control, with more cost and effort falling on you. The colour of each rung shows the direction of the trade-off, not a score.

THE AI DEPLOYMENT LADDER

Where your model runs decides who controls your data

Seven ways to run an AI model. Convenience and power at the top, data control and cost-of-effort at the bottom.

↑ MORE CONVENIENT · LESS CONTROL

FRONTIER VENDOR MODELS

1 Vendor chat app

Type into Claude / ChatGPT / Gemini. Consumer plans may train on your text.

Least control

2 Vendor model via API

Your software calls the model. Business terms: generally no training.

3 Vendor model inside your cloud

Same model, run in your AWS / Azure / GCP account, in your region.

Vendor + your cloud

4 Shared self-hosted model

You host an open model and serve many customers. Tenants share infra.

Open model

5 Inside the client's cloud

Self-hosted model in one client's private cloud. Data never leaves it.

6 On the client's premises

Runs on their own hardware, in their building. Nothing leaves the site.

7 On the local device

Model runs on one laptop. No cloud, nothing uploaded. Smaller model is the trade.

Most control

↓ MORE CONTROL · MORE EFFORT & COST

OPEN MODELS YOU HOST

The move most teams miss: **you don't pick one**. Route high-volume simple work to the low-control end, heavy work to a stronger backend. Deployment is a routing decision, not an either-or.

For years the debate was which rung to stand on. That is the wrong question. **The future is hybrid.** You do not pick a rung, you route across the ladder: high-volume, rudimentary work runs cheaply and privately at the low-control end, even on-device, while anything that needs real reasoning power is sent to a stronger model higher up. A single product spans several rungs at once, choosing per task based on cost, data sensitivity, and how much power the job actually needs.

WHAT HYBRID LOOKS LIKE IN PRACTICE: THE ARCHITECT-BUILDER PATTERN

The clearest recent signal of this shift comes from Google. In work announced in September 2026 around their Antigravity SDK, they demonstrate an **Architect-Builder** pattern that combines cloud-model scale with local-model advantages: a cloud model acts as the **architect**, the planner and conductor, while a swarm of local, on-device models act as the **builders**, handling the heavy lifting entirely on the device.

That is exactly the hybrid architecture described here, made concrete. One model directs, many execute, and the system deliberately spans the top and bottom of the ladder at the same time. When a frontier lab ships this as a first-class pattern, it is a strong sign that hybrid orchestration is becoming the default, not a niche optimisation. The direction the industry is moving, and the direction of this framework, are converging.

Why hybrid is becoming the default

- **Data sovereignty is tightening.** More work has to stay in a specific region, cloud, or building, which pushes real workloads down the ladder and forces per-task placement decisions.
- **Open models are now genuinely capable.** The low-control end of the ladder is no longer a compromise. Small, on-device models are good enough to own the high-volume, routine work outright.
- **The cost gap is too large to ignore.** Running everything on frontier models versus routing intelligently is a difference no serious operator will leave on the table.

One subtlety that trips people up

The ladder actually mixes two questions: who owns the model, and where it runs. The top rungs use a vendor's model, the bottom rungs use an open model you host. That is why a shared self-hosted setup is not automatically more private than a vendor model running inside your own cloud. Hybrid design means reasoning about both axes at once, ownership and location, not just one. This is also, in spirit, the client-server split of an earlier computing era returning in a new form: a light, local tier for the common case, a powerful remote tier for the hard case.

The teams that win will stop asking "API or self-host," and start designing deployment as an architecture, a hybrid routing layer that sends each task to the right rung. Within a couple of years, almost every serious deployment will be the second kind.